ZeroCollab: P2P Project Management and Assessment Tool

Songül Nur Öztürk, Özlem Nur Bora, Nisanur Karataş
Department of Computer Engineering
Istanbul Kultur University, Istanbul, Turkiye

***Abstract*—**This paper presents ZeroCollab, a browser-based agile project management tool we built as our capstone project. The main goal was to keep the app usable under unstable or unavailable network conditions, so we built it around offline-first storage with peer-to-peer synchronization. Data lives in IndexedDB [5][7] on each browser and is shared directly between peers using PeerJS [2] over Web Real-Time Communication (WebRTC) [1], with Supabase [3] handling authentication and backup. As an additional experimental capability layer, we added a token-based feature management module using Hedera Token Service [10] on testnet, where a small ZC token balance gates Free, Plus, and Premium feature tiers. We also built a Large Language Model (LLM)-assisted evaluation module that scores uploaded project deliverables against ontology-defined quality criteria. The app includes a four-level backlog (Epic, Feature, User Story, Task), sprint planning with readiness checks, a Kanban board, roadmap, dashboard, messaging, calendar, and a basic video call module. In two-browser tests, updates between peers arrived in under two seconds, and Mirror Node balance lookups averaged around 1.3 seconds. The main limitation is that two edits to the same field within ~100 ms can collide under our timestamp merge rule, which we plan to address with CRDTs.

***Keywords*—**Agile, P2P, WebRTC, IndexedDB, Offline-First, React, Supabase, Hedera, Token-Based Features, Ontology, LLM-Assisted Evaluation

I. INTRODUCTION

Jira [9], Trello, Monday.com, and Linear are widely used agile tools, but their sprint boards and task lists become unreachable when the internet connection is unavailable. For teams working under unstable or intermittent network conditions, this is a real friction point.

Cost is another concern. Vendor-hosted data means accepting their pricing and terms. Per-seat licensing makes most platforms too expensive for student teams.

Kleppmann et al. [6] proposed the local-first model: keep data on the device first, sync over the network when possible. WebRTC [1] fits this since it allows direct browser-to-browser transfers.

Our project, ZeroCollab, follows this approach. Everything is stored locally in IndexedDB [5][7]. Peers share data through PeerJS [2]/WebRTC. Supabase [3] provides login and backup. On top of this core we experimented with a token-based feature management layer using Hedera [10]—a small ZC token balance gates which features each user can access. The rest of the paper covers the proposed architecture, synchronization mechanism, token layer, implementation details, and evaluation results obtained during testing under unstable network conditions.

II. RELATED WORK

Jira [9] has full Scrum/Kanban support but is cloud-only and expensive. Trello covers basic Kanban without sprints. Monday.com has automation but no offline mode. Linear caches read-only data offline. Azure DevOps targets enterprise CI/CD with Microsoft accounts. Table I summarizes the comparison.

CRDTs (Yjs, Automerge) handle concurrent edits at the data layer but are libraries, not finished apps. We use timestamp-based merging for now and plan to add CRDTs later.

TABLE I. COMPARISON WITH EXISTING TOOLS

Feature	Jira	Trello	Monday	Linear	ZeroCollab
Sprint Plan.	Yes	No	Partial	Yes	Yes
Kanban	Yes	Yes	Yes	Yes	Yes
Backlog	Yes	No	Partial	Yes	Yes
Roadmap	Yes	No	Yes	Yes	Yes
P2P Sync	No	No	No	No	Yes
Offline	No	No	No	Limited	Yes
Cloud	Yes	Yes	Yes	Yes	Yes
Token Econ.	No	No	No	No	Yes
Meetings	No	No	No	No	Proto.

III. METHODOLOGY

A. System Architecture

The architecture is organized into a frontend layer, an application logic layer, and a data layer that combines P2P communication, local persistence, and cloud backup (Fig. 1). React [4] 19 with Vite 7 and Tailwind CSS 4 forms the frontend. The logic layer handles state, auth, and the update engine.

PeerJS [2] wraps WebRTC [1] data channels for peer communication. Peer IDs combine a stored base with a random suffix to avoid reconnection collisions. Session Traversal Utilities for NAT (STUN) and Traversal Using Relays around NAT (TURN) servers assist Network Address Translation (NAT) traversal. IndexedDB [5] via idb [7] stores data locally. Supabase [3] handles email authentication, PostgreSQL backup, and recovery for peers that have been offline for a long time.

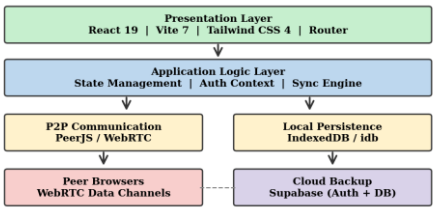


Fig. 1. Layered architecture of ZeroCollab.

B. Update Propagation

Changes go to local IndexedDB first, then get broadcast to peers through PeerJS (Fig. 2). Receivers compare timestamps—newer records get applied, older ones get ignored. Tombstone flags prevent deleted items from reappearing via stale copies. Supabase gets the same update in the background.

If two users change the exact same field within about 100 ms, one edit can be silently lost. During testing we saw the concurrent merge time fluctuate between 800 and 1200 ms depending on how close the edits were. This is the main thing we want to fix by adding CRDTs.

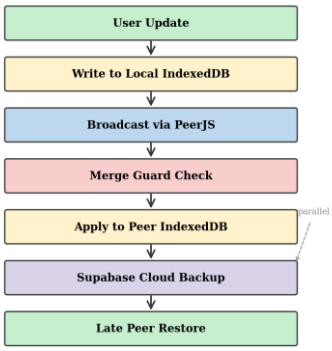


Fig. 2. Step-by-step flow of how an update travels from one peer to another and to the cloud backup.

C. Application Modules

The Product Backlog supports four hierarchy levels (Epic → Feature → User Story → Task) with Create, Read, Update, and Delete (CRUD) operations and peer sharing. The Roadmap has Now/Next/Later buckets for prioritization.

Sprint Planning (Fig. 3) has a readiness checklist we added—it verifies at least one user story is committed with linked tasks, and that the sprint has a goal and dates. When you close a sprint, a modal shows any unfinished items and lets you move them back to the backlog.

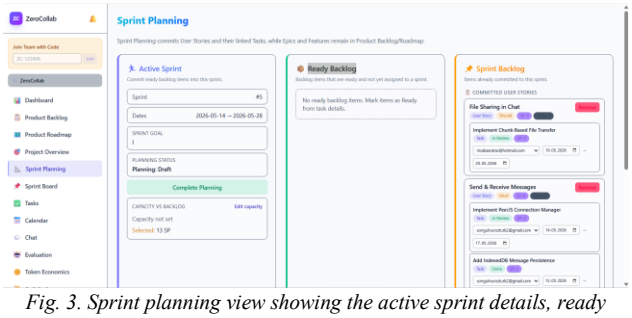


Fig. 3. Sprint planning view showing the active sprint details, ready backlog, and committed items.

The Tasks board (Fig. 6) has four Kanban columns with drag-and-drop. The Dashboard (Fig. 7) displays task stats, workload per member, and recent activity with Recharts [8]. We set up the Calendar to only show Tasks and Meetings—Epics, Features, and User Stories are planning items so we kept them out of the schedule view. Chat handles channels and DMs stored in IndexedDB and shared via PeerJS. The Meeting Room does basic WebRTC video calls but we have not tested it across many different network setups so it is still a prototype. An activity log records user actions and synchronization events across the workspace.

D. Token-Based Capability System

As an experimental capability layer separate from the core sync work, we created a ZC token on Hedera Testnet using the Hedera Token Service (HTS) [10]. The app reads each user's balance through the Mirror Node Representational State Transfer (REST) Application Programming Interface (API) (Fig. 4). This part of the project is an early prototype rather than a production-ready blockchain integration.

The balance maps to one of three tiers: Free (0 ZC), Plus, or Premium. Each tier has different limits—Free users can create up to two projects, and roadmap editing is gated. The tier check and quota enforcement happen on the client using the cached balance, which keeps the behavior consistent with the rest of our local-first design.

Admin-triggered token distribution uses a basic operator-signing flow on testnet. Transfers themselves usually take 2–3 seconds, but the Mirror Node can lag a few seconds behind the actual ledger, so there is a noticeable gap before a recipient's tier updates in the user interface (UI). Users currently connect by pasting their Hedera account ID manually, which is not the cleanest onboarding. We treat the whole token layer as an experimental module rather than something we would ship to production as-is.

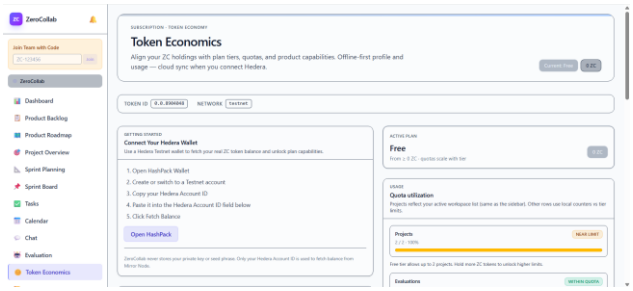


Fig. 4. Token economics page showing wallet connection, plan tier, and quota usage.

E. Ontology-Based Evaluation

A team can upload project deliverables—a source archive, a PDF, or a DOCX—and the application scores them against quality dimensions using a large language model. The key design choice is that the rubric is not hard-coded: quality dimensions, scoring rules, and submission scopes are read from an OWL/Turtle ontology at runtime, so the same dimension set drives every assessment and new criteria can be added by editing the ontology rather than the source code. Before calling the model, the app selects the most relevant files across categories such as tests, configuration, and domain folders, keeping the request within budget while covering the whole project. Each evaluation returns a weighted score with per-dimension feedback, is scoped to the active project, and is shared

across the team, while a separate activity log records project actions for auditing. The model first selects which of the twelve quality dimensions apply, so a design document is not penalized for missing code. Each is scored out of ten, weighted through the ontology, and mapped to a letter grade. The evaluation workflow was designed to support collaborative review scenarios inside the same workspace. Uploaded deliverables are linked to the active project context so evaluation history can be revisited later by team members. The same rubric categories can also be reused across different project submissions, so the team applies a consistent assessment standard without rewriting the underlying evaluation logic. Evaluation summaries are synchronized through the same local-first synchronization layer used by the other modules. This keeps assessment results available even after temporary offline usage.

F. Technologies

TABLE II. TECHNOLOGY STACK

Category	Technology	Role
Frontend	React 19 + Vite 7	SPA build tooling
Styling	Tailwind CSS 4	Utility-first UI
P2P	PeerJS / WebRTC	Peer communication
Storage	IndexedDB / idb 8	Local persistence
Cloud	Supabase 2	Auth & backup
Charts	Recharts 3	Dashboard charts
Blockchain	Hedera HTS	Token economy
Testing	Vitest 3	Unit/integration tests
PWA	vite-plugin-pwa	Offline caching

IV. EXPERIMENTAL RESULTS

A. Peer Sync Timing

We tested with two browser windows on the same local network. Each scenario ran five times and we averaged the results. For the offline test we disabled the network in DevTools, made changes, then reconnected.

TABLE III. PEER SYNC TEST RESULTS

Scenario	Avg. Time	Consistency	Result
Backlog creation	~780 ms	Stable	PASS
Backlog deletion	~860 ms	Stable	PASS
Sprint task update	~720 ms	Stable	PASS
Roadmap creation	~830 ms	Stable	PASS
Concurrent merge	~1150 ms	Variable*	PASS*
Offline reconnect	~1900 ms	Minor lag	PASS

** Concurrent merge times ranged from ~800 to ~1200 ms across trials. Edits to the same field within ~100 ms may lose one update.*

Fig. 5 shows the results. Regular operations like creating a backlog item or updating a task status all came in under one second. The offline reconnect was slowest because of the Supabase round-trip. Concurrent merges had the most variance.

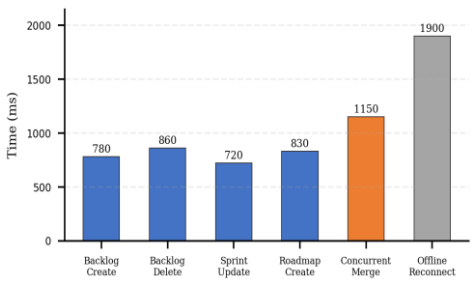


Fig. 5. Average propagation times across six test scenarios, measured in milliseconds.

B. Token Layer Testing

TABLE IV. TOKEN ECONOMY TEST RESULTS

Scenario	Avg. Time	Status	Result
Balance fetch (Mirror)	~1300 ms	Stable	PASS
Tier upgrade trigger	~450 ms	Stable	PASS
Quota enforcement	~180 ms	Stable	PASS
Roadmap lock (Free)	Instant	Stable	PASS
Admin token send	~2800 ms	Variable**	PASS

*** Admin transfers depend on Testnet load. Mirror Node refresh sometimes lagged 2–4 seconds behind the ledger.*

Mirror Node balance fetches were mostly consistent but got slower occasionally when Testnet was busy. Tier changes and quota enforcement worked correctly every time. Admin transfers were the least predictable—Testnet response ranged from about 1.5 to 3 seconds, and the Mirror Node needed extra time on top of that to show the updated balance.

C. Module Status

Every main module has been implemented and tested: Backlog, Roadmap, Sprint Planning (with readiness checklist and close-sprint validation), Tasks board, Dashboard, Messaging, Calendar, Settings, Token Economics, Evaluation, Activity Log, and PWA offline mode. The Meeting Room works but is still a prototype. Our Vitest suite spans 36 test files with 456 test cases covering backlog validation, IndexedDB persistence, merge logic, token tier and quota rules, ontology loading, and UI utilities. All covered tests pass.

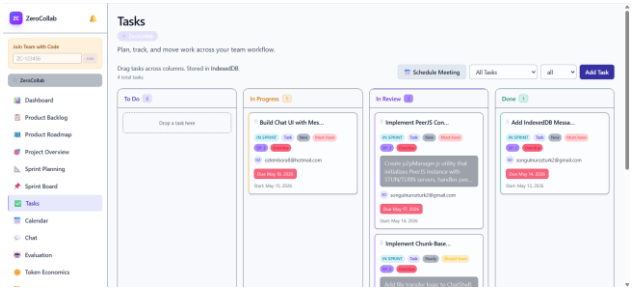


Fig. 6. Tasks board showing four Kanban columns (To Do, In Progress, In Review, Done) with task cards and status tags.

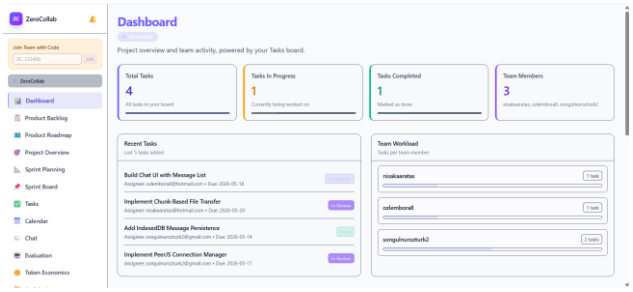


Fig. 7. Dashboard displaying total task counts, team workload breakdown, and recent task activity.

V. CONCLUSIONS AND FUTURE WORK

By the end of the project we had a working agile tool that does not depend on a central server staying up. Peer sync, local storage, and cloud backup hold together as a single system, and the Hedera token layer adds something we have not seen in mainstream agile tools—though it is fair to call this layer experimental, especially around wallet

onboarding and the balance refresh delays we saw on testnet.

Peer sync held up across every scenario we tested, and the common operations all finished in under a second. The timestamp merge handles normal editing fine, but the ~100 ms collision window where one edit can be silently dropped is a real limitation, and the proper fix is CRDTs.

Next steps on our list: swap the timestamp merge for CRDTs, automate wallet connection so users do not paste account IDs by hand, improve meeting room stability across different NAT setups, extend burndown and velocity analytics, and extend the ontology-based evaluation system with richer project assessment capabilities.

ACKNOWLEDGMENT

The authors thank their advisor, Yusuf Altunel, for his guidance throughout this project. This work also makes use of the results of the Ontology Based Software Engineering and Zero Time Software Development studies carried out by Yusuf Altunel within the Department and at LC Waikiki, which informed the ontology-driven design of the evaluation and configuration layers.

REFERENCES

[1] W3C, “WebRTC 1.0: Real-time Communication Between Browsers,” W3C Recommendation, Jan. 2021. [Online]. Available: https://www.w3.org/TR/webrtc/

[2] PeerJS Contributors, “PeerJS documentation,” 2024. [Online]. Available: https://peerjs.com/docs

[3] Supabase, “Supabase documentation,” 2024. [Online]. Available: https://supabase.com/docs

[4] Meta Open Source, “React documentation,” 2024. [Online]. Available: https://react.dev

[5] Mozilla Developer Network, “IndexedDB API,” 2024. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/IndexedDB_API

[6] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan, “Local-first software: You own your data, in spite of the cloud,” in Proc. Onward!, 2019, pp. 154–178.

[7] J. Archibald, “IndexedDB with usability (idb library),” 2022. [Online]. Available: https://github.com/jakearchibald/idb

[8] Recharts Team, “Recharts documentation,” 2024. [Online]. Available: https://recharts.org

[9] Atlassian, “Jira software documentation,” 2024. [Online]. Available: https://support.atlassian.com/jira-software-cloud/

[10] Hedera, “Hedera Token Service documentation,” 2024. [Online]. Available: https://docs.hedera.com/hedera/sdks-and-apis/sdks/token-service